



Questões Práticas

ATENÇÃO: Esta atividade não vale nota. Ela foi elaborada apenas como material de revisão e estudo para a Prova II.

1. Sistema de Funcionários com Classe Abstrata e Polimorfismo

Implemente, em Java, uma hierarquia de classes para representar funcionários de uma empresa, utilizando **classe abstrata**, **herança** e **polimorfismo**.

Crie uma classe abstrata chamada **Funcionario**.

Essa classe deve possuir os seguintes atributos privados:

```
private String nome;  
private double salarioBase;
```

A classe **Funcionario** deve possuir, no mínimo:

- a) Construtor padrão;
- b) Construtor parametrizado;
- c) Métodos **get** e **set** para os atributos;
- d) Método concreto **exibirDados()**;
- e) Método abstrato **calcularSalario()**.

O método abstrato deve possuir a seguinte assinatura:

```
public abstract double calcularSalario();
```

Em seguida, crie as classes **Gerente**, **Desenvolvedor** e **Estagiario**, que devem herdar de **Funcionario**.

A classe **Gerente** deve possuir:

- a) Atributo privado **bonus**;
- b) Construtores;
- c) Métodos **get** e **set**;
- d) Implementação do método **calcularSalario()**, retornando **salarioBase + bonus**.

A classe **Desenvolvedor** deve possuir:

- a) Atributo privado **horasExtras**;
- b) Construtores;

- c) Métodos `get` e `set`;
- d) Implementação do método `calcularSalario()`, considerando que cada hora extra vale R\$ 50,00.

A classe `Estagiario` deve possuir:

- a) Atributo privado `auxilioTransporte`;
- b) Construtores;
- c) Métodos `get` e `set`;
- d) Implementação do método `calcularSalario()`, retornando `salarioBase + auxilioTransporte`.

Requisitos:

- a) A classe `Funcionario` deve ser abstrata;
- b) A classe `Funcionario` não deve ser instanciada diretamente;
- c) Todas as subclasses concretas devem implementar `calcularSalario()`;
- d) Os atributos devem ser privados;
- e) Valores negativos devem ser tratados com valores padrão;
- f) Usar `@Override` nos métodos sobrescritos.

No método `main`, crie um vetor do tipo `Funcionario` contendo objetos das classes `Gerente`, `Desenvolvedor` e `Estagiario`. Depois, percorra esse vetor chamando o método `exibirDados()`.

Exemplo de uso esperado:

```
Funcionario[] funcionarios = {
    new Gerente("Ana", 5000.0, 1500.0),
    new Desenvolvedor("Carlos", 3000.0, 10),
    new Estagiario("Marina", 1200.0, 300.0)
};

for (Funcionario f : funcionarios) {
    f.exibirDados();
}
```

2. Sistema de Pagamentos com Interface

Implemente, em Java, um sistema simples de pagamentos utilizando **interfaces** e **polimorfismo**.

Crie uma interface chamada `Pagamento`, contendo os seguintes métodos:

```
void processar(double valor);
String getDescricao();
```

Em seguida, crie as classes `CartaoCredito`, `Boleto` e `Pix`, que devem implementar a interface `Pagamento`.

A classe `CartaoCredito` deve possuir:

- a) Atributo privado `numeroCartao`;
- b) Construtores;
- c) Métodos `get` e `set`;
- d) Implementação do método `processar()`;
- e) Implementação do método `getDescricao()`.

A classe `Boleto` deve possuir:

- a) Atributo privado `codigoBarras`;
- b) Construtores;
- c) Métodos `get` e `set`;
- d) Implementação do método `processar()`;
- e) Implementação do método `getDescricao()`.

A classe `Pix` deve possuir:

- a) Atributo privado `chavePix`;
- b) Construtores;
- c) Métodos `get` e `set`;
- d) Implementação do método `processar()`;
- e) Implementação do método `getDescricao()`.

Crie também uma classe chamada `ProcessadorPagamento`, com o seguinte método:

```
public void executarPagamento(Pagamento pagamento, double valor
)
```

Esse método deve receber qualquer objeto que implemente `Pagamento`, exibir a descrição do meio de pagamento e chamar o método `processar()`.

Requisitos:

- a) Criar a interface `Pagamento`;
- b) Implementar a interface em `CartaoCredito`, `Boleto` e `Pix`;
- c) Não utilizar herança entre as classes de pagamento;
- d) Usar o método `executarPagamento()` de forma polimórfica;
- e) Impedir pagamento com valor menor ou igual a zero;
- f) Usar `@Override` nos métodos implementados.

No método `main`, teste o sistema realizando pagamentos com cartão de crédito, boleto e Pix.

Exemplo de uso esperado:

```
ProcessadorPagamento processador = new ProcessadorPagamento();

processador.executarPagamento(new CartaoCredito("1234"), 200.0)
;
processador.executarPagamento(new Boleto("999888777"), 150.0);
processador.executarPagamento(new Pix("email@teste.com"), 75.0)
;
```

3. Figuras Geométricas com Classe Abstrata, Interface e Exceção

Implemente, em Java, um sistema de figuras geométricas utilizando **classe abstrata**, **interface**, **polimorfismo** e **tratamento de exceções**.

Crie uma classe abstrata chamada **Forma**, contendo os seguintes atributos privados:

```
private String cor;
private int x;
private int y;
```

A classe **Forma** deve possuir:

- a) Construtor padrão;
- b) Construtor parametrizado;
- c) Métodos **get** e **set**;
- d) Método concreto **exibirDados()**;
- e) Método abstrato **calcularArea()**.

O método abstrato deve possuir a seguinte assinatura:

```
public abstract double calcularArea();
```

Crie também uma interface chamada **Desenhavel**, contendo:

```
void desenhar();
```

Depois, crie as classes **Circulo**, **Retangulo** e **Triangulo**. Todas devem herdar de **Forma** e implementar **Desenhavel**.

A classe **Circulo** deve possuir:

- a) Atributo privado **raio**;
- b) Construtores;
- c) Métodos **get** e **set**;
- d) Implementação de **calcularArea()**;
- e) Implementação de **desenhar()**.

A classe **Retangulo** deve possuir:

- a) Atributos privados **largura** e **altura**;
- b) Construtores;

- c) Métodos `get` e `set`;
- d) Implementação de `calcularArea()`;
- e) Implementação de `desenhar()`.

A classe `Triangulo` deve possuir:

- a) Atributos privados `base` e `altura`;
- b) Construtores;
- c) Métodos `get` e `set`;
- d) Implementação de `calcularArea()`;
- e) Implementação de `desenhar()`.

Crie uma exceção personalizada chamada `MedidaInvalidaException`.

Essa exceção deve ser lançada quando alguma medida da figura for menor ou igual a zero.

Requisitos:

- a) Criar a classe abstrata `Forma`;
- b) Criar a interface `Desenhavel`;
- c) Criar a exceção personalizada `MedidaInvalidaException`;
- d) Usar `throw` para lançar a exceção;
- e) Usar `throws` nos métodos necessários;
- f) Tratar a exceção no método `main` com `try-catch`;
- g) Usar polimorfismo com um vetor do tipo `Forma`.

No método `main`, crie diferentes figuras, armazene-as em um vetor do tipo `Forma` e chame os métodos `exibirDados()` e `calcularArea()`.

4. Sistema de Animais com Herança, Interfaces e Polimorfismo

Implemente, em Java, um sistema para representar diferentes animais, utilizando **herança, interfaces e polimorfismo**.

Crie uma classe abstrata chamada `Animal`, contendo:

- a) Atributos privados `nome` e `idade`;
- b) Construtor padrão;
- c) Construtor parametrizado;
- d) Métodos `get` e `set`;
- e) Método concreto `apresentar()`;
- f) Método abstrato `emitirSom()`.

Crie as seguintes interfaces:

```
public interface Voador {
```

```
        void voar();  
    }
```

```
    public interface Nadador {  
        void nadar();  
    }
```

```
    public interface Corredor {  
        void correr();  
    }
```

Depois, crie as classes **Cachorro**, **Passaro**, **Peixe** e **Pato**.

A classe **Cachorro** deve:

- a) Herdar de **Animal**;
- b) Implementar **Corredor**;
- c) Sobrescrever **emitirSom()**;
- d) Implementar **correr()**.

A classe **Passaro** deve:

- a) Herdar de **Animal**;
- b) Implementar **Voador**;
- c) Sobrescrever **emitirSom()**;
- d) Implementar **voar()**.

A classe **Peixe** deve:

- a) Herdar de **Animal**;
- b) Implementar **Nadador**;
- c) Sobrescrever **emitirSom()**;
- d) Implementar **nadar()**.

A classe **Pato** deve:

- a) Herdar de **Animal**;
- b) Implementar **Voador**, **Nadador** e **Corredor**;
- c) Sobrescrever **emitirSom()**;
- d) Implementar **voar()**, **nadar()** e **correr()**.

Requisitos:

- a) A classe **Animal** deve ser abstrata;
- b) **Animal** não deve ser instanciada diretamente;
- c) Todas as subclasses devem sobrescrever **emitirSom()**;
- d) Uma classe pode implementar mais de uma interface;

- e) Usar `@Override`;
- f) Evitar métodos vazios sem necessidade.

No método `main`, crie um vetor de `Animal` com objetos de diferentes subclasses. Percorra o vetor chamando `apresentar()` e `emitirSom()`.

Depois, teste separadamente os comportamentos das interfaces, chamando `voar()`, `nadar()` e `correr()` nos animais adequados.

5. Sistema Bancário com Exceções Personalizadas

Implemente, em Java, um sistema simples de conta bancária utilizando **encapsulamento** e **tratamento de exceções**.

O sistema deve possuir as seguintes classes:

```
ContaBancaria
SaldoInsuficienteException
ValorInvalidoException
Main
```

A classe `ContaBancaria` deve possuir os seguintes atributos privados:

```
private String titular;
private double saldo;
```

A classe `ContaBancaria` deve possuir:

- a) Construtor padrão;
- b) Construtor parametrizado;
- c) Métodos `get` e `set`;
- d) Método `depositar(double valor)`;
- e) Método `sacar(double valor)`;
- f) Método `exibirSaldo()`.

Crie uma exceção personalizada chamada `SaldoInsuficienteException`.

Essa exceção deve ser lançada quando o usuário tentar sacar um valor maior que o saldo disponível.

Crie também uma exceção personalizada chamada `ValorInvalidoException`.

Essa exceção deve ser lançada quando:

- a) O valor do depósito for menor ou igual a zero;
- b) O valor do saque for menor ou igual a zero;
- c) O saldo inicial informado for negativo.

Requisitos:

- a) Criar as exceções personalizadas;
- b) Usar `extends Exception`;

- c) Usar **throw** para lançar as exceções;
- d) Usar **throws** nos métodos que podem gerar erro;
- e) Tratar as exceções no **main** com **try-catch**;
- f) Exibir mensagens claras para o usuário;
- g) Exibir o saldo final.

Exemplo de saída esperada:

```
Saldo inicial: R$ 100.0
Depositando R$ 50.0...
Saldo atual: R$ 150.0

Tentando sacar R$ 300.0...
Erro: saldo insuficiente.

Programa finalizado normalmente.
```

6. Revisão Teórica e Análise de Código

Responda às questões abaixo com suas próprias palavras. Quando necessário, utilize pequenos exemplos em Java.

a) Sobrecarga x Sobreposição

Explique a diferença entre **sobrecarga** e **sobreposição** de métodos.

Na resposta, informe:

- a) Quando cada uma acontece;
- b) Se ocorre em tempo de compilação ou em tempo de execução;
- c) Se os parâmetros precisam ser iguais ou diferentes;
- d) Um exemplo simples de cada caso.

b) Classe Abstrata x Interface

Explique a diferença entre **classe abstrata** e **interface**.

Na resposta, informe:

- a) Quando usar classe abstrata;
- b) Quando usar interface;
- c) Se podem ter atributos;
- d) Se podem ter construtores;
- e) Como são usadas em Java: **extends** ou **implements**.

c) Problema do Diamante

Explique o que é o **Problema do Diamante** e por que Java não permite herança múltipla de classes.

Depois, explique como as interfaces ajudam a resolver esse problema.

d) throw x throws

Explique a diferença entre **throw** e **throws**.

Na resposta, informe:

- a) Onde cada um é usado;
- b) Qual deles lança a exceção;
- c) Qual deles apenas declara que o método pode gerar uma exceção;
- d) Um exemplo simples.

e) Checked x Unchecked Exceptions

Explique a diferença entre exceções **checked** e **unchecked**.

Na resposta, cite pelo menos dois exemplos de cada uma.

f) Análise de Código

Analise o código abaixo e explique o que será impresso na tela:

```
abstract class Animal {
    public abstract void emitirSom();

    public void apresentar() {
        System.out.println("Animal se apresentando...");
        emitirSom();
    }
}

class Cachorro extends Animal {
    @Override
    public void emitirSom() {
        System.out.println("Au au!");
    }
}

class Gato extends Animal {
    @Override
    public void emitirSom() {
        System.out.println("Miau!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Cachorro();
        Animal a2 = new Gato();

        a1.apresentar();
        a2.apresentar();
    }
}
```

Explique também por que esse código é um exemplo de polimorfismo.

7. Desafio: Sistema de Notificações com Interface, Lambda e Exceção

Implemente, em Java, um sistema de notificações utilizando **interface**, **polimorfismo**, **lambda** e **tratamento de exceções**.

Crie uma interface chamada `Notificavel`, contendo:

```
void enviar(String mensagem);
```

Crie as classes `Email`, `SMS` e `PushNotification`, que devem implementar `Notificavel`.

Cada classe deve exibir uma mensagem diferente ao enviar a notificação.

Crie uma classe `ServicoNotificacao`, contendo o método:

```
public void notificar(Notificavel meio, String mensagem)
```

Esse método deve receber qualquer objeto que implemente `Notificavel` e chamar o método `enviar()`.

Crie também uma exceção personalizada chamada `MensagemVaziaException`.

Essa exceção deve ser lançada quando a mensagem for nula ou vazia.

Requisitos:

- criar a interface `Notificavel`;
- criar as classes `Email`, `SMS` e `PushNotification`;
- criar a classe `ServicoNotificacao`;
- criar a exceção personalizada `MensagemVaziaException`;
- usar `try-catch` no método `main`;
- criar pelo menos uma notificação usando expressão `lambda`.

Exemplo de lambda esperado:

```
Notificavel console = mensagem -> System.out.println("Console: " + mensagem);
```

No método `main`, envie notificações por `Email`, `SMS`, `PushNotification` e pela implementação com `lambda`.